Numerical Methods for Non-Linear Mechanics

Viet Anh Quach

December 12, 2025

1 Introduction

This short report present the code that is assign for the subject "Numerical Methods for Non-Linear Mechanics" of the Master Program for Civil Engineering, teached by professor Stafano-Dal Pont at Laboratory 3SR, University Grenoble Alpes, France. The object of the course is studying the non-linear problems in mechanics, caused by both the geometric and material behavior. The assigment practices therefore is to build in the most general form of the constitutive law, which is not linearity. 2 main problems are takin into account: A console problem in 2D and a Truss problem in 3D, hence the remaining of this reports are divided into two sections of the former subject. Besides, this is a ideal chance to practice, thus I also try to provide a solution for the problem of Finite Element Method in 1D and 2D. The main code is actually written in `C++`, with libraries mostly building from scratches. They have been upload to `Github`, the link to download is provided in section 6. Any corrections and comments are highly appreciated. I would like to thanks the professors for letting me, even though being a PhD student, could join this program to learn. I really appreciated and enjoyed the class.

2 Theory

The following problem was fully described in lecture course [1] of professor DAL-PONT. This section just represent it but with the particular methods using within the following codes and the reasons of these choice.

2.1 Console

The whole geometry is described in fig. 1. Briefly speaking, the simple truss structure composed by 3 nodes linked by 2 bars and external force F applied on the node C maing an angel θ with the vertical axis, causing the displacement of the bars and nodes. Gravity is not taken into account. In this case, $\vec{x}$ representing position of node C is considered as the main unknown, owing to the fact that it is the only free node and fully represent the equilibrium position of the system. $\hat{e}^b$ is the unit vector of the corresponding bar b, which is calculated by the position vector of the node where the bar b begins ($\vec{x}^B$) and

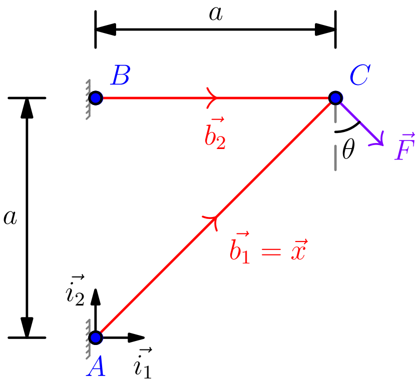


Figure 1: Two elements truss: console 2D

ends $(\vec{x}^E)$:

$$\vec{b} = \vec{x}^{E(b)} - \vec{x}^{O(b)} \tag{2.1a}$$

$$l^b = \|\vec{b}\| \tag{2.1b}$$

$$\hat{e}^b = \frac{\vec{b}}{l^b} \tag{2.1c}$$

In truss elements, the only internal effort considered is the axial stress and further, also is a constant. To satisfy the nodal equilibrium, at position C the sum of the internal force (left-hand side) and the external force (right-hand side) must be balanced:

$$\sum_{b=1}^{2} N^b = N^1 + N^2 = F \tag{2.2}$$

which N^b denotes this component that the right part exerts on the left part of bar b, which is assumed to depend only on the length of bar l^b via the constitutive law:

$$N^b = \mathcal{A}^b(l^b) \tag{2.3}$$

In case of large displacement, this relation is no longer linear, as the bar's vectors are changed accordingly with the new position of the nodes, in our case is particularly node C. The following elastic constitutive laws may be considered, so that the tension N^b taken into account only on the bar length variation:

- Linear law:

$$\mathcal{A}^b(l^b) = \alpha^b \frac{l^b - l_0^b}{l_0^b} \tag{2.4}$$

- Saint-Venant Kirchhoff law:

$$\mathcal{A}^b(l^b) = \alpha^b \frac{(l^b)^2 - (l_0^b)^2}{2(l_0^b)^2} \tag{2.5}$$

- Logarithmic law:

$$\mathcal{A}^b(l^b) = \alpha^b ln(\frac{l^b}{l_0^b}) \tag{2.6}$$

Hence, in general form, the problem of the console could be written as follows:

$$\begin{aligned} \text{Given the force } \vec{\mathcal{F}}, \\ \text{Find the position } \vec{x} \text{ of node C such that,} \\ -\mathcal{A}^1(l^1(\vec{x}))\hat{e}^1(\vec{x}) - \mathcal{A}^2(l^2(\vec{x}))\hat{e}^2(\vec{x}) + \vec{\mathcal{F}} = \vec{0} \end{aligned} \tag{2.7}$$

This problem is solved by Newton's iterative method in vector form for solving systems of equations. For the order-one truncated Taylor series expansion, expression at the iteration (k+1) shall be:

$$\underline{\underline{\nabla \mathcal{F}}}(\underline{x}^{(k)})\delta(\underline{x}^{(k)}) = -\underline{\mathcal{F}}(\underline{x}^{(k)}) \tag{2.8}$$

Solving the linear system get us the new value of δx, so we could update the position for the next iteration by:

$$\underline{x}^{(k+1)} = \underline{x}^{(k)} + \delta \underline{x}^{(k)} \tag{2.9}$$

Apply to the console problem eq. (2.8) becomes The method requires a stopping criterion: eq. (2.10) is chosen to be implemented in the code, as it satisfies the force equilibrium condition at nodes of the problem.

$$\left| f\left(x^{(k)}\right) \right| < \epsilon \tag{2.10}$$

where ϵ is a number small enough to be considered as 0, relatively to the problem solving. Another substitute approch is stopping at the iteration where the displacement is neglectable, which will be used in section 2.2:

$$\left| x^{k+1} - x^k \right| < \epsilon \tag{2.11}$$

The object is now to transform equation eq. (2.3) into a linear form eq. (2.8) so that it could be numerically solved. Thanks to first-order Taylor expansion again, we obtain the development of sum of the force, which should be approching 0, at iteraion `k+1` (or in another word, at when node **C** would take an increment in position into $\vec{x} + \delta\vec{x}$):

$$\vec{\mathcal{F}}(\vec{x} + \delta\vec{x}) = \vec{\mathcal{F}}(\vec{x}) + \left(\nabla \vec{\mathcal{F}}(\vec{x})\right) @\delta\vec{x} = 0 \tag{2.12}$$

where the Jacobian matrix $\nabla \vec{\mathcal{F}}(\vec{x})$ is described in the following euqation using Einsten's sum notation:

$$\nabla \vec{\mathcal{F}}(\vec{x}) = -\frac{d\mathcal{A}^i}{dl^i}\left(\vec{e}^i(\vec{x}) \otimes \vec{e}^i(\vec{x})\right) - \frac{\mathcal{A}^i\left(l^i(\vec{x})\right)}{l^i(\vec{x})}\left(\mathbb{I} - \vec{e}^i(\vec{x}) \otimes \vec{e}^i(\vec{x})\right) \tag{2.13}$$

Finally, we can solve the system using a single linear vector equation, which is suitable to be implemented into computational calculating:

$$\left(\nabla \vec{\mathcal{F}}(\vec{x}^k)\right) @\delta\vec{x}^k = -\vec{\mathcal{F}}(\vec{x}^k) \tag{2.14}$$

The first iteration starts from the initial condition of the system, then the iteration begins, until $\vec{\mathcal{F}}(\vec{x}^k) < \epsilon$ to be considered null, as discussed above.

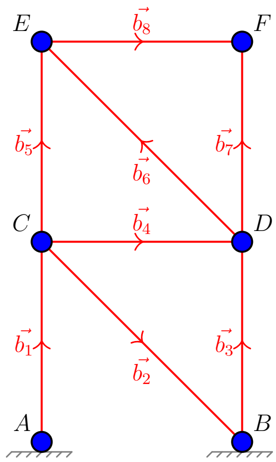


Figure 2: 3D truss

2.2 Truss

Similarly in the previous problem, each bar's geometry is fully accessible through the position of nodes. Their vectors could be determined totally similar in the previous problem via eq. (2.1). The problem of the small deformation truss can be stated as:

$$\begin{aligned} &\text{Given the external force } \vec{F}^{e/n}, n \in \mathcal{N}_F \text{ and } \vec{U}^n, n \in \mathcal{N}_I \\ &\text{find the displacement vector } \vec{u}^n, n \in \mathcal{N} \text{ so that } \vec{u}^n = \vec{U}^n, n \in \mathcal{N}_I \text{ and} \\ &\forall \vec{v}^n \text{ such that } \vec{v}^n = 0, n \in \mathcal{N}_I \\ &\sum_{b\in\mathcal{B}} k^b \vec{e}^{\,b}_0 \left(\vec{u}^{E(b)} - \vec{u}^{O(b)}\right) \vec{e}^{\,b}_0 \left(\vec{u}^{E(b)} - \vec{u}^{O(b)}\right) = \sum_{n\in\mathcal{N}_F} \vec{F}^{e/n} \vec{v}^n \end{aligned} \tag{2.15}$$

which expressed in matrix notation as:

$$\forall \underline{\mathcal{V}}^t \underline{\underline{\mathcal{K}}} \underline{\mathcal{U}} = \underline{\mathcal{V}}^t \underline{\mathcal{F}} \tag{2.16}$$

or in the simpliest way:

$$\underline{\underline{\mathcal{K}}}\,\underline{\mathcal{U}} = \underline{\mathcal{F}} \tag{2.17}$$

We can easily verify that:

$$\underline{\mathcal{F}} = \begin{bmatrix} F_1^{e/1} \\ F_2^{e/1} \\ F_3^{e/1} \\ \vdots \\ F_1^{e/N} \\ F_2^{e/N} \\ F_3^{e/N} \end{bmatrix} ; \quad \underline{\mathcal{U}} = \begin{bmatrix} u_1^1 \\ u_2^1 \\ u_3^1 \\ \vdots \\ u_1^N \\ u_2^N \\ u_3^N \end{bmatrix} \tag{2.18}$$

Within each component the subscript $k = 1, 2, 3$ stands for the 3 directions in 3D and the superscript $n = 1..N$ determines indices of nodes.

Meanwhile, the global stiffness matrix $\underline{\underline{\mathcal{K}}}$ could be assembled by multiple methods. One of them is by using the elementary application K^b of bar b along with the connectivity matrix $\underline{\underline{C^n}}$ as followed:

$$\underline{\underline{K}} = \sum_{b \in \mathcal{B}} \left(\underline{\underline{C}}^{E(b)} - \underline{\underline{C}}^{O(b)} \right) \underline{\underline{K}}^b \left(\underline{\underline{C}}^{E(b)} - \underline{\underline{C}}^{O(b)} \right) \tag{2.19}$$

$$\underline{\underline{C}}^n = \left[\begin{array}{ccc|ccc|ccc} 0 & \cdots & 0 & 1 & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & 0 & 1 & 0 & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 0 & 0 & 1 & 0 & \cdots & 0 \end{array}\right] \tag{2.20}$$

$$\underbrace{\qquad\qquad}_{3(n-1)} \quad \underbrace{\qquad\quad}_{3} \quad \underbrace{\qquad\qquad}_{3(N-n)} \tag{2.21}$$

Though this method is easily implemented in coding small problem, when calculating manually by hand, we prefer using the elementary stiffness matrix

$$\underline{\underline{K}}^b = \underline{\underline{K}}^b \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \tag{2.22}$$

At this point, $\underline{\underline{K}}$ should be singular i.e $\det(\underline{\underline{K}}) = 0$ because the kinematic boundary condition is not yet applied. Two different techniques are usually proposed: the direct method and the approximated method (so-called penalization method). The approximated is obtained by adding $1/\epsilon$ to the main diagonal terms of $\underline{\underline{K}}$ which corresponds to the nodes of $\mathcal{N}_I$:

$$\underline{\underline{K}}^{\epsilon}_{ii} = \underline{\underline{K}}_{ii} + \frac{1}{\epsilon}; i = 3(n-1) + k; n \in \mathcal{N}_I, k = 1, 2, 3 \tag{2.23}$$

2.3 FEM1D

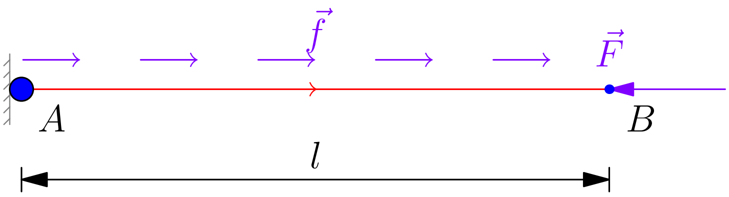


Figure 3: 1D Finite Element

The equilibrium equations therefore, described in the strong form:

$$\frac{dN}{dx} + f = 0; \tag{2.24a}$$
$$N(0) = -R; \tag{2.24b}$$
$$N(l) = F; \tag{2.24c}$$
$$N = EA\frac{du}{dx}; \tag{2.24d}$$
$$u(0) = 0; \tag{2.24e}$$

The weak form is expressed by;

$$\forall v \in V, \int_0^l EA\frac{du}{dx}\frac{dv}{dx}dx - Rv(0) = \int_0^l fvdx + Fv(l) \tag{2.25}$$

$$\forall v \in V, a(u, v) + b(R, v) = L(v) \tag{2.26}$$

rewritten in tensor notation:

$$\text{Find } \underline{\mathcal{U}} \text{ and } \mathcal{R} \text{ such that:} \tag{2.27}$$

$$U^1 = 0; \tag{2.28}$$

$$\underline{\underline{\mathcal{K}}}\underline{\mathcal{U}} + \mathcal{R} = \underline{\mathcal{F}} \tag{2.29}$$

2.4 FEM2D

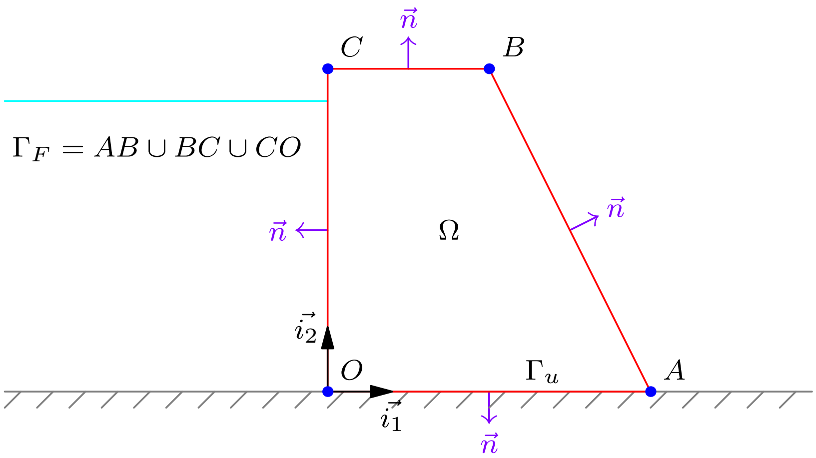


Figure 4: 2D Finite Element

$$\forall \vec{x} \in \Omega, \mathrm{div}\sigma + \vec{f} = 0; \tag{2.30}$$

$$\forall \vec{x} \in \Omega, \sigma = \lambda \mathrm{tr}\epsilon(\vec{u})\mathbb{I} + 2\mu\epsilon(\vec{u}); \tag{2.31}$$

$$\forall \vec{x} \in \Gamma_u, \vec{u} = \vec{U}; \tag{2.32}$$

$$\forall \vec{x} \in \Gamma_F, \sigma @ \vec{n} = \vec{F}; \tag{2.33}$$

$$\forall \vec{x} \in \Gamma_u, \sigma @ \vec{n} = \vec{R}; \tag{2.34}$$

$$\tag{2.35}$$

weak form:

$$\text{Find } \vec{u} \in V_0 \text{ and } \vec{\mathcal{R}} \text{ such that:} \tag{2.36}$$

$$\forall \vec{v} \in V, \int_\Omega (\lambda \mathrm{tr}\epsilon(\vec{u})\mathrm{tr}\epsilon(\vec{v}) + 2\mu\epsilon(\vec{u}) : \epsilon(\vec{v}))ds - \int_{\Gamma_u} \vec{R}\vec{v}dl = \int_\Omega \vec{f}\vec{v}ds + \int_{\Gamma_F} \vec{F}\vec{v}dl; \tag{2.37}$$

3 Validation

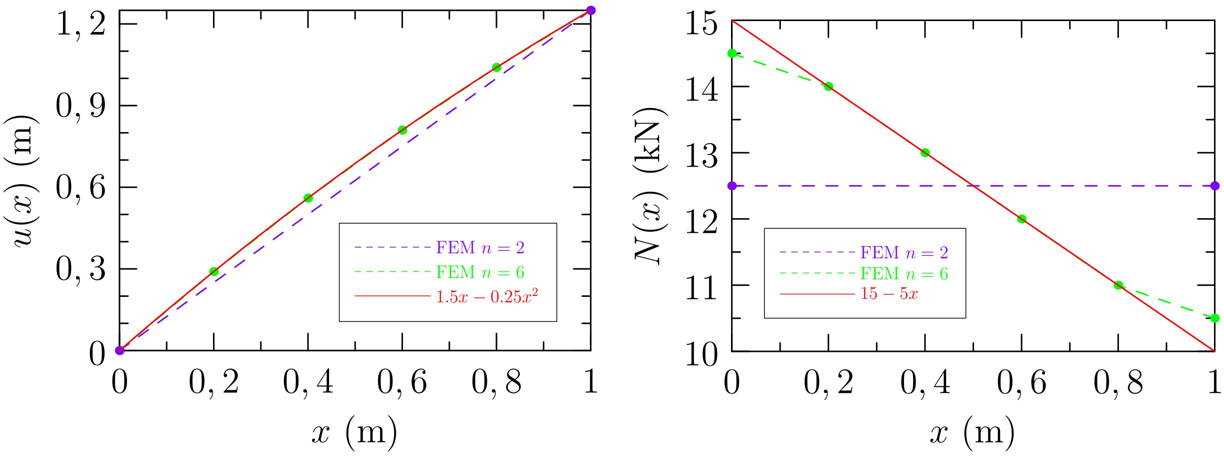
3.1 Console

As proved in figure, the rate of convergence by Newton method is quadratic. Student choosed the Saint-Venant Kirchhoff law to present the result, with the intention to capture the non-linearity in both the constitutive law via eq. (2.3) and the large deformation's effect in eq. (2.1).

3.2 Truss

3.3 FEM1D

Taking trivial value as follows: $f = 5kN$; $F = 10kN$; $EA = 10kN$; $L = 1m$, once could verify the anylytic solution to be: $R = -15kN$; $u(x) = 1.5x - 0.25x^2(m)$; $N(x) = 15 - 5x(kN)$; Using the script, though the reaction force at the first node $R(x = 0) = -15$kN is always calculated precisely, the deformation $u(x)$ and the axial force $N(x)$ are highly depended on the number of nodes. Figure 5 shows the numerical solution of the code, comparing with the analytical one. It was shown that the deformation $u(x)$ is obviously more precise, when the bar is discretized by using more elements. The particular point of `FEM` comparing with Galerkin's method is using special shape function, particularly in this case is first-order hat function, via Lagrange's basis. This is well explained by when we discretized $u(x) = N_j u_j$. As u_j is a constant, $u(x)$ is a first order polynomial (fig. 5a), leading to its derivative to be a constant, hence $N(x) = EA\ du/dx$ is a constant on each element also (fig. 5b).



(a) Deformation

(b) Axial Force

Figure 5: Solutions of the 1D FEM problem

3.4 FEM2D

In constructing...Stucking at the generation of the mesh.....

4 Application Conclusion

4.1 Console

The code is able to handle 2D Truss problems with 2 nodes, as a trial solution to the 3D truss problem following.

4.2 Truss

The code is able to handle any non-linear truss problem, 2D and 3D, including the console problem above. Though the Hardening incremented law should be implemented in the future to wider the range of problem that could be handled.

4.3 FEM1D

The code has been implemented in a way to solve any 1D bar problmem, with whatever the constraint conditions are, both Dirichlet and Neuman's condition. As long as the differential equation satisfied the form $u_{xx} = f(x) + c$ with $f(x)$ is a polynomial function of x, including cx^0. This leads to the application of distributed load and concentration load exhibited on the bar. Though the numerical cost could be reduced strongly using reference element, element Matrix, ... etc but this initial version is to test out all the functioning of the sub-coded also. Gladly, the solution seems to adapted well with the analytical one as expected.

5 Conclusion

In a nutshell, the code provided trustable result. Further implemented still rested to continue. Thanks to using C++, time elapsed to execute one code is less then 10^{-3} second on the student's computer. Further optimization...

6 Annexxe

Full version of the codes, including libraries used, could be downloaded in this link.

References

[1] DAL-PONT, S. (2020). *Numerical Methods for Non-Linear Mechanics.* Université Grenoble Alpes, Grenoble, France. 1